

REPUBLIQUE DEMOCRATIQUE DU CONGO
MINISTERE D'ENSEIGNEMENT SUPERIEUR ET UNIVERSITAIRE
UNIVERSITE CATHOLIQUE DE BUKAVU



FACULTE DE SCIENCES / BAC-1 INFO

CAHIER DE CHARGES PROJET TUTOIRE

PRESENTATEURS :

NSHOBOLÉ MARUME IMMACULÉE

NSHOBOLÉ MUFUNGIZI ESTHER

NZOGU ABELI NGILA DIEUDONNE PLAMEDI

OGISHA NSHOMBO JOSUE

OJOMBO FEZA DANIELLA

ANNEE ACADEMIQUE 2024-2025

SYSTÈME DE GESTION DES ENTRÉES À L'UNIVERSITÉ PAR SCANING DE QR CODE

1. Introduction

Dans un souci d'améliorer la gestion des flux d'étudiants, la sécurité et la traçabilité des activités académiques, ce projet propose le développement d'un système de contrôle d'accès basé sur des QR codes.

Chaque étudiant disposera d'un QR code personnel scanné à l'entrée principale, mais aussi à l'entrée de chaque salle (auditoires, laboratoires, salles d'examen, etc.).

Ce système permettra :

- De valider les identités pour accéder au campus ou aux salles ;
- De **signer automatiquement les présences aux cours** ;
- De **conditionner l'accès** à certains lieux en fonction de critères académiques ou financiers (paiement des frais).

2. État de besoin

Dans le but d'améliorer la sécurité et la traçabilité des mouvements au sein de l'Université Catholique de Bukavu, ce système de gestion des accès par QR code répond aux besoins suivants :

- **Identification numérique automatisée** : Chaque étudiant est doté d'un QR code unique, généré automatiquement lors de son enregistrement, associé à son identifiant universitaire.
- **Contrôle d'accès différencié** selon les zones :
 - À l'**entrée principale** de l'université, tout étudiant inscrit et reconnu par le système se voit accorder l'accès. En revanche, tout QR code non valide ou inexistant dans la base de données entraîne un refus automatique.
 - En **période d'examen**, l'accès aux salles est soumis à une vérification supplémentaire : l'étudiant doit être en ordre de paiement pour pouvoir entrer.
 - Dans les **auditoires**, le scan du QR code permet d'enregistrer automatiquement la présence de l'étudiant, servant ainsi de preuve de participation aux cours.
- **Traçabilité et historique des passages** : Le système enregistre chaque scan avec précision (porte concernée, date, heure, statut). Ces données permettent d'établir des feuilles de présence automatiques, consultables par l'administration.
- **Interface Web intuitive pour les utilisateurs** : Une application Web permettra d'effectuer le scan en toute simplicité via un bouton dédié. À la sélection de ce bouton, la caméra de l'appareil est activée pour scanner le QR

code, et une réponse claire s'affiche à l'écran ("Accès autorisé" ou "Accès refusé"), selon l'état de l'étudiant et l'emplacement scanné.

- **Espace administrateur sécurisé** : L'administrateur accède au système via une authentification (login et mot de passe). Il peut gérer les inscriptions (ajouter, modifier, désactiver un étudiant), consulter l'historique des scans, et exporter des rapports journaliers ou spécifiques à un point d'accès. Ces rapports sont générés sous format PDF non modifiable pour garantir leur intégrité.
- **Extensibilité du système** : Le système est conçu pour s'adapter à d'autres usages à l'avenir, tels que la gestion d'accès aux bibliothèques, laboratoires, restaurants universitaires, ou encore la gestion de la présence au sein d'activités extrascolaires.

3. Analyse fonctionnelle

3. Analyse fonctionnelle

Le système de gestion des accès par QR code repose sur l'interaction entre plusieurs **acteurs** et un ensemble de **fonctions principales** décrites ci-dessous.

3.1 Acteurs du système

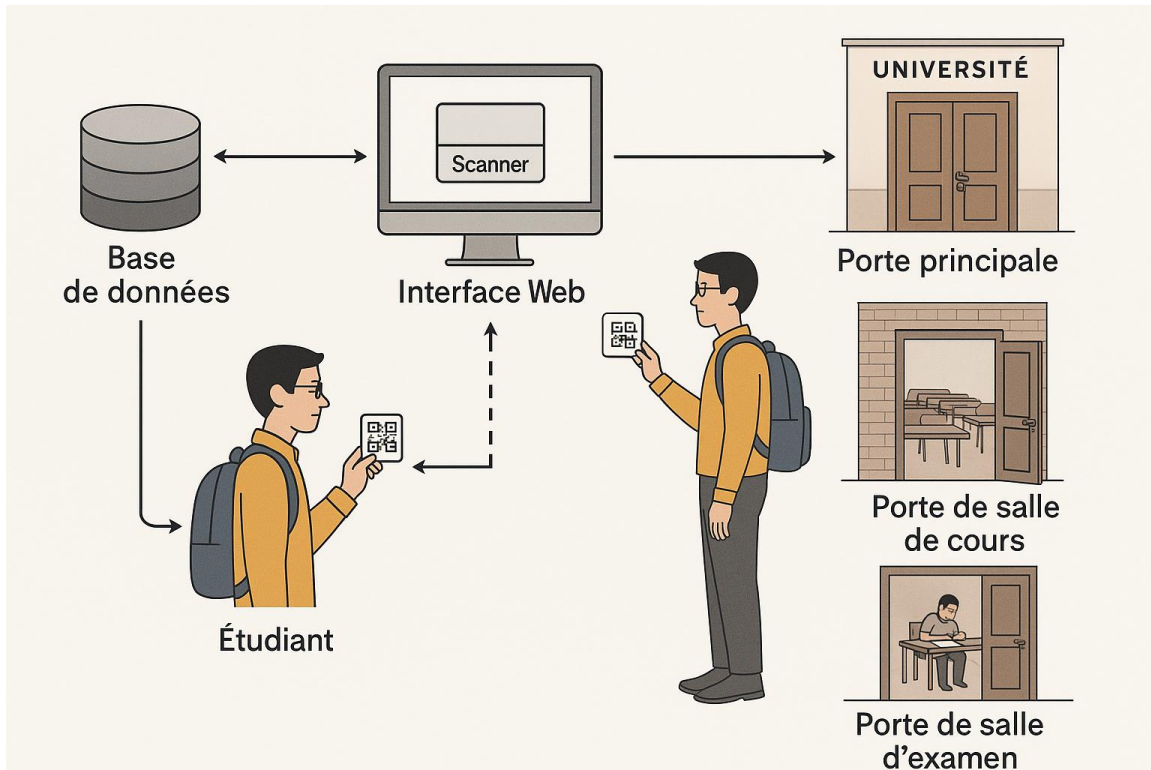
- **Étudiant** :
 - Possède une carte QR code personnelle.
 - Présente son QR code à l'entrée principale, aux auditoriums ou aux salles d'examen.
 - Ne dispose d'aucun droit administratif dans le système.
- **Administrateur** :
 - Gère la base de données des étudiants (ajout, modification, désactivation).
 - Supervise les accès et la traçabilité via l'historique des scans.
 - Possède un compte sécurisé avec identifiants (login, mot de passe).
 - Peut exporter des rapports (journaliers ou spécifiques à un point d'accès) sous format PDF.
 - L'administrateur peut ajouter, consulter, modifier et valider les paiements des étudiants (montant, motif, date, statut), afin de conditionner l'accès à certaines zones.
 - Gère la base de données des étudiants (ajout, modification, désactivation), ainsi que les comptes des opérateurs de scan (création, approbation, désactivation).

3.2 Cas d'utilisation principaux

Acteur	Cas d'utilisation	Description détaillée
Étudiant	Scanner son QR code	L'étudiant se rend à un point d'accès, scanne son QR code, reçoit un message en retour.
	Accéder à un local	L'accès est accordé ou refusé selon les règles (existence, période d'examens, frais).
Administrateur	Authentification	L'admin se connecte via login/mot de passe pour accéder à son espace personnel.
	Gestion des étudiants	L'admin peut ajouter un nouvel étudiant, modifier ses informations ou désactiver son compte.
	Suivi des accès	L'admin consulte l'historique des scans par date, lieu ou étudiant.
	Exportation de rapports	Il est possible d'exporter les rapports d'accès en format PDF sécurisé.
	Approuver/Gérer les opérateurs de scan	L'administrateur peut consulter les demandes de création de compte pour les opérateurs de scan, approuver ou refuser ces comptes, et modifier leurs informations ou les désactiver.

3.3 Fonctions principales du système

- Lecture du QR code via caméra ou scanner.
- Comparaison du code scanné avec les données enregistrées dans la base.
- Vérification des règles d'accès (valide/invalidé, période, statut administratif).
- Enregistrement de chaque passage dans l'historique.
- Affichage d'un message clair (autorisé/refusé).
- Interface Web intuitive selon le profil de l'utilisateur (verificateur ou admin).
- Le système permettra aux administrateurs de saisir, visualiser et mettre à jour les informations de paiement des étudiants (montant, motif, date, statut), et de générer des rapports sommaires sur l'état des paiements.
- Le système permettra aux administrateurs de créer des comptes pour les agents de scan, de définir leur rôle, et de gérer leur statut (en attente d'approbation, approuvé, désactivé) avant qu'ils ne puissent se connecter et utiliser l'interface de scan.



4. Architecture technique

Le système proposé repose sur une architecture distribuée combinant des technologies web et des composants matériels accessibles. Son objectif est de garantir une interaction fluide entre les étudiants et le système de contrôle d'accès tout en assurant la fiabilité et la sécurité des données traitées.

4.1. Composants principaux

- **Base de données MySQL**
Héberge les informations relatives aux étudiants (identité, QR code, état de régularité) ainsi que les historiques d'accès par porte, date et heure.
- **Application Web**
Développée avec **Flask (Python)**, cette interface permettra :
 - Le scan du QR code via la **caméra d'un PC** ou **webcam externe**.
 - La visualisation et la gestion des historiques.
 - La connexion sécurisée des administrateurs (authentification par mot de passe).
- **Module de Scan**
Le scan est réalisé par la **caméra intégrée** ou un **scanner QR externe**, configuré pour reconnaître le QR code d'un étudiant et en vérifier la validité dans la base.

- **Points d'accès multiples**

Le système est déployé sur trois niveaux :

- **Entrée principale** : accès général.
- **Porte de salle de cours** : pour le suivi des présences.
- **Porte de salle d'examen** : filtrage basé sur la régularité financière.

4.2. Flux de fonctionnement (mise à jour)

1. L'étudiant présente son QR code devant la caméra ou le scanner.
2. L'image est capturée et décodée par l'application web.
3. Une requête est envoyée à la base de données afin de vérifier si l'identifiant QR est associé à un étudiant existant.
4. En cas de correspondance, les règles d'accès sont évaluées selon le point de contrôle :
 - À l'entrée principale : tout étudiant reconnu a droit d'accès.
 - À la porte d'examen : l'accès est accordé uniquement si l'étudiant est en ordre de paiement.
5. Une réponse est immédiatement affichée à l'écran :
 - **Accès autorisé à [Nom Prénom]** — avec l'horodatage, le point d'entrée et un fond vert.
 - **Accès refusé pour [Nom Prénom]** — avec la raison (inconnu, irrégularité administrative, etc.) et un fond rouge.
6. Chaque tentative (réussie ou non) est automatiquement enregistrée dans la table des accès avec tous les détails nécessaires.

4.3. Sécurité et intégrité

- Les administrateurs accèdent à la plateforme via un espace sécurisé.
- L'accès au système pour les opérateurs de scan est conditionné par une approbation préalable par un administrateur, garantissant que seuls les personnels autorisés peuvent utiliser les interfaces de scan.
- Les fichiers de présence générés sont **non modifiables** (export en PDF ou CSV verrouillé).
- Les données sont sauvegardées régulièrement et peuvent être restaurées en cas de défaillance.

4.3. Génération et accès du QR code

Pour l'administrateur :

- Lors de l'ajout d'un nouvel étudiant, le système génère le QR code et affiche les informations de l'étudiant ainsi que le QR code visuellement comme une carte.
- Les boutons restent : "Imprimer la carte" (pour l'impression physique) et "Enregistrer".

- **Ajout** : Une fois l'étudiant enregistré, le système déclenchera **automatiquement l'envoi d'un email** à l'adresse email de l'étudiant. Cet email contiendra le QR code de l'étudiant sous forme d'image ou en pièce jointe PDF, accompagné de ses informations principales.

❓ Pour l'étudiant (en cas de perte ou d'oubli de la carte physique) :

- L'étudiant pourra consulter son email (depuis son smartphone ou tout autre appareil connecté).
- Il pourra alors présenter le QR code affiché sur son écran de téléphone au scanner pour les contrôles d'accès.

5. Conception de la base de données

La base de données du système est construite pour assurer la traçabilité des accès, la gestion des étudiants, la validation des présences et le contrôle des paiements. Elle est structurée autour des entités suivantes :

5.1. Tables principales

a) etudiants

Contient les informations de chaque étudiant.

- id (PK)
- nom
- postnom
- prenom
- genre
- faculte
- promotion
- email
- qr_code (valeur encodée générée automatiquement)

b) acces

Enregistre chaque tentative d'accès à une porte.

- id (PK)
- id_etudiant (FK vers etudiants.id)
- id_porte (FK vers portes.id)
- id_utilisateur(FK vers utilisateurs_systemes.id)
- timestamp (date et heure de scan)
- statut (valeurs possibles : "autorisé", "refusé")

- raison_refus (texte libre, ex : "Paiement non à jour")

c) portes

Liste les points d'entrée contrôlés par QR code.

- id (PK)
- nom (ex: "Entrée principale", "Auditoire 2", "Salle examen A")
- type_porte (valeurs possibles : "générale", "auditoire", "examen")

d) cours

- id (PK) : Identifiant unique du cours.
- titre_cours (VARCHAR) : Nom complet du cours (ex: "Bases de Données", "Introduction à la Programmation").
- faculte (VARCHAR) : Faculté à laquelle le cours est rattaché (ex: "Informatique", "Sciences Commerciales").
- promotion (VARCHAR) : Promotion/Année d'étude à laquelle le cours est destiné (ex: "L1", "G2").
- description (TEXT, optionnel) : Courte description du cours.

e) presences

Permet de générer automatiquement des listes de présence par porte et par jour.

- id (PK)
- id_etudiant (FK vers etudiants.id)
- id_cours (FK vers cours.id)
- id_porte (FK vers portes.id)
- date_presence (DATE)
- heure_scan

f) paiements

Enregistre les paiements effectués par chaque étudiant.

- id (PK)
- id_etudiant (FK vers etudiants.id)
- montant
- motif (ex : frais académique)
- date_paiement

g) administrateurs

Stocke les comptes d'administration.

- id (PK)
- nom_utilisateur
- mot_de_passe_hash (mot de passe sécurisé)

h) utilisateurs_systeme

- id (PK)
- nom_utilisateur
- mot_de_passe_hash
- nom_complet
- role_specifique (VARCHAR, optionnel) : Préciser le rôle (ex: "Sécurité entrée principale", "Surveillant auditoire", "Surveillant examen", « CP de l'auditoire »).
- statut (VARCHAR) : Statut du compte de l'agent (ex: 'en_attente', 'approuve', 'desactive').

5.2. Relations entre les tables

Relation	Type	Description
etudiants → acces	1 à N	Un étudiant peut scanner plusieurs fois dans différentes portes.
etudiants → presences	1 à N	Permet de générer les listes de présence par salle.
portes → acces	1 à N	Chaque accès est lié à une porte spécifique.
portes → presences	1 à N	Chaque présence est liée à une porte précise (auditoire ou salle).
etudiants → paiements	1 à N	Permet de vérifier l'état de paiement avant de permettre l'accès à un exam.
cours → presences	1 à N	Un cours peut avoir plusieurs enregistrements de présence (plusieurs étudiants y assistent, ou le même étudiant y assiste plusieurs fois). Chaque enregistrement de présence est lié à un cours spécifique.
utilisateurs_systeme → acces	1 à N	Un utilisateur peut effectuer plusieurs scans. Chaque enregistrement d'accès est lié à l'agent qui l'a effectué.

5.3. Contraintes et sécurités

- **Clés primaires** sur toutes les tables.
- **Clés étrangères** pour relier les entités (avec ON DELETE CASCADE là où pertinent).
- **Hash des mots de passe** pour les administrateurs.
- **Validation côté backend** pour vérifier les conditions d'accès (paiement, identité, période d'examen).

5.4. Vue réelle de la base de données

Voici comment est construite la base de données effectuée sous MySQL :

```
CREATE DATABASE IF NOT EXISTS qr_access;
USE qr_access;

-- =====
-- 1. Table etudiants
-- =====
CREATE TABLE IF NOT EXISTS etudiants (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(50) NOT NULL,
    postnom VARCHAR(50) NOT NULL,
    prenom VARCHAR(50) NOT NULL,
    genre ENUM('M', 'F') NOT NULL,
    faculte VARCHAR(50) NOT NULL,
    promotion VARCHAR(50) NOT NULL,
    email VARCHAR(50) UNIQUE NOT NULL,
    qr_code TEXT NOT NULL COMMENT 'Valeur encodée du QR code généré'
);

-- =====
-- 2. Table portes
-- =====
CREATE TABLE IF NOT EXISTS portes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(50) NOT NULL COMMENT 'Nom du point d'entrée (ex: Entrée principale,
    Auditoire 2, Salle examen)',
    type_porte ENUM('generale', 'auditoire', 'examen') NOT NULL COMMENT 'Type de
    porte pour application des règles d'accès'
);

-- =====
-- 3. Table cours
-- =====
CREATE TABLE IF NOT EXISTS cours (
    id INT AUTO_INCREMENT PRIMARY KEY,
    titre_cours VARCHAR(50) NOT NULL,
    faculte VARCHAR(50) NOT NULL,
    promotion VARCHAR(50) NOT NULL,
    description_cours TEXT
);
```

```

-- =====
-- 4. Table administrateurs
-- =====
CREATE TABLE IF NOT EXISTS administrateurs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom_utilisateur VARCHAR(50) UNIQUE NOT NULL,
    mot_de_passe_hash VARCHAR(50) NOT NULL COMMENT 'Mot de passe haché pour la sécurité'
);
-- =====
-- 5. Table utilisateurs_systeme
-- =====
CREATE TABLE IF NOT EXISTS utilisateurs_systeme (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom_utilisateur VARCHAR(50) UNIQUE NOT NULL,
    mot_de_passe_hash VARCHAR(50) NOT NULL,
    nom_complet VARCHAR(50) NOT NULL,
    role_specifique VARCHAR(50) COMMENT 'Ex: Sécurité entrée principale, Surveillant examen, CP de l\'auditoire',
    statut ENUM('en_attente', 'approuve', 'desactive') DEFAULT 'en_attente' NOT NULL
COMMENT 'Statut du compte (nécessite approbation admin)'
);
-- =====
-- 6. Table paiements
-- =====
CREATE TABLE IF NOT EXISTS paiements (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_etudiant INT NOT NULL,
    montant DECIMAL(10, 2) NOT NULL,
    motif VARCHAR(50) NOT NULL COMMENT 'Ex: frais académique',
    date_paiement DATE NOT NULL,
    statut ENUM('valide', 'en_attente', 'rejete', 'partiel') DEFAULT 'en_attente' NOT NULL
COMMENT 'Statut du paiement pour vérification d éligibilité',
    FOREIGN KEY (id_etudiant) REFERENCES etudiants(id) ON DELETE CASCADE
);
-- =====
-- 7. Table presences
-- =====
CREATE TABLE IF NOT EXISTS presences (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_etudiant INT NOT NULL,
    id_cours INT NOT NULL,
    id_porte INT NOT NULL,
    date_presence DATE NOT NULL,
    heure_scan TIME NOT NULL,
    FOREIGN KEY (id_etudiant) REFERENCES etudiants(id) ON DELETE CASCADE,
    FOREIGN KEY (id_cours) REFERENCES cours(id) ON DELETE RESTRICT, -- On ne supprime pas
un cours s il y a des présences liées
    FOREIGN KEY (id_porte) REFERENCES portes(id) ON DELETE RESTRICT -- On ne supprime pas
une porte s il y a des présences liées
);

```

```

-- =====
-- 8. Table acces
-- =====
CREATE TABLE IF NOT EXISTS acces (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_etudiant INT NOT NULL,
    id_porte INT NOT NULL,
    timestamp DATETIME NOT NULL COMMENT 'Date et heure complètes du scan',
    statut ENUM('autorise', 'refuse') NOT NULL,
    raison_refus TEXT COMMENT 'Raison si l'accès est refusé',
    id_utilisateur_systeme INT COMMENT 'Agent ayant effectué le scan',
    FOREIGN KEY (id_etudiant) REFERENCES etudiants(id) ON DELETE CASCADE,
    FOREIGN KEY (id_porte) REFERENCES portes(id) ON DELETE RESTRICT, -- Empêche la
suppression d'une porte si des accès y sont liés
    FOREIGN KEY (id_utilisateur_systeme) REFERENCES utilisateurs_systeme(id) ON DELETE SET
NULL -- Si l'utilisateur est supprimé, son ID dans les accès devient NULL
);

=====
--          Application des requettes les plus importantes et souvent utilisees
=====

-- 1. INSERT : Ajouter des étudiants et un cours
INSERT INTO etudiants (nom, postnom, prenom, genre, faculte, promotion, email, qr_code)
VALUES ('Kabongo', 'Mutombo', 'Junior', 'M', 'Informatique', 'Bac1',
'junior.kabongo@ucb.cd', 'qr_code_12345');

INSERT INTO etudiants (nom, postnom, prenom, genre, faculte, promotion, email, qr_code)
VALUES ('Mukwege', 'Kalume', 'Esther', 'F', 'Informatique', 'Bac1', 'esther.mukwege@ucb.cd',
'qr_code_23456');

INSERT INTO etudiants (nom, postnom, prenom, genre, faculte, promotion, email, qr_code)
VALUES ('Bahati', 'Mugisho', 'Josué', 'M', 'Informatique', 'Bac1', 'josue.bahati@ucb.cd',
'qr_code_34567');

INSERT INTO cours (titre_cours, faculte, promotion, description_cours)
VALUES ('Mathématiques discrètes', 'Informatique', 'Bac1', 'Cours fondamental sur les
structures discrètes, les ensembles, les graphes, etc.');
```

```

-- 2. SELECT + WHERE + ORDER BY : Ici on selection les étudiants de la faculté informatique
et on les ordonne par ordre alphabetique de leurs noms
SELECT nom, postnom, prenom, faculte
FROM etudiants
WHERE faculte = 'Informatique'
ORDER BY nom ASC;
```

```

-- 3. UPDATE : Modification de l'adresse e-mail d'un étudiant
UPDATE etudiants
SET email = 'junior.kabongo2025@ucb.cd'
WHERE id = 1;
```

```

-- 4. DELETE : Supprimer un cours
DELETE FROM cours
WHERE titre_cours = 'Mathématiques discrètes';
```

```
-- 5. GROUP BY : Grouper les étudiants par promotion
```

```
SELECT promotion  
FROM etudiants  
GROUP BY promotion;
```

```
-- 6. INNER JOIN : Afficher le nom de chaque étudiant et les montants qu'il a payés
```

```
SELECT e.nom, e.postnom, p.montant, p.date_paiement  
FROM etudiants e  
JOIN paiements p ON e.id = p.id_etudiant;
```

```
-- 8. ALTER : Ajouter un champ téléphone aux étudiants
```

```
ALTER TABLE etudiants  
ADD COLUMN telephone VARCHAR(20);
```

```
/* ALERTE :
```

Les insertions manuelles ci-dessous sont uniquement destinées à la démonstration et le test pour ce travail pratique.

Conformément à la logique de notre projet, la base de données doit rester vide de toute donnée lors de sa première utilisation réelle.

Ces données seront donc supprimées par la suite. */

6. Planning prévisionnel

Le développement du système s'étalera sur une durée de **sept semaines**, selon un découpage en phases cohérentes visant à garantir une progression méthodique du projet. Le tableau ci-dessous présente les grandes étapes de réalisation :

Semaine	Phase	Objectifs
Semaine 1	Lancement & Documentation	- Présentation du sujet à l'équipe et à l'encadrant - Collecte des exigences fonctionnelles et non fonctionnelles - Rédaction du cahier des charges
Semaine 2	Conception UML & Base de données	- Élaboration des diagrammes UML (cas d'utilisation, classes, séquence) - Définition des entités et relations dans la base de données - Mise en place de la structure MySQL
Semaine 3	Développement du backend (Flask)	- Mise en place de l'environnement de développement - Implémentation des API Flask pour le scan, la validation et la gestion des accès - Connexion à la base de données MySQL
Semaine 4	Développement du frontend (Web)	- Conception de l'interface Web de scan et d'administration - Intégration de la caméra pour lecture QR code - Mise en place de l'affichage des messages de réponse (accès autorisé/refusé)
Semaine 5	Gestion des utilisateurs & sécurité	- Intégration de l'espace administrateur avec système de login - Téléchargement sécurisé des rapports journaliers (PDF non modifiable) - Mise en place des règles d'accès selon le type de porte et le statut de paiement
Semaine 6	Tests & Intégration	- Scénarios de test par cas d'usage - Simulation des accès réels à différentes portes - Correction des bugs et ajustements finaux
Semaine 7	Finalisation & Présentation	- Rédaction du rapport final du projet - Préparation des supports visuels pour la soutenance (diapos, démonstration) - Présentation devant l'encadreur ou jury

7. Conclusion

Le système de gestion des entrées à l'Université Catholique de Bukavu par scan de QR code constitue une solution technologique novatrice qui répond à un besoin croissant de **sécurité, traçabilité** et **efficacité** dans le contrôle d'accès au sein de l'institution.

Au-delà de la simple ouverture de la porte principale, ce projet étend sa portée jusqu'aux accès des auditoriums et des salles d'examen, permettant non seulement de **valider les identités en temps réel**, mais aussi de **générer automatiquement des listes de présence** et de **vérifier l'éligibilité des étudiants** selon leur situation académique et financière.

Ce projet constitue une opportunité concrète pour les membres de l'équipe de :

- Mettre en pratique leurs compétences en **développement web, base de données** et **programmation backend**,
- Apprendre à **travailler en équipe de façon collaborative**,
- Découvrir les **enjeux réels liés à l'automatisation des systèmes d'information**.

Grâce à une planification rigoureuse et une répartition équilibrée des tâches, même les membres débutants pourront contribuer activement au projet tout en consolidant leurs bases techniques. À terme, cette application offrira une solution adaptable et évolutive que l'université pourra exploiter à plus grande échelle.